

Modification de Listes: Complications

Qu'est-ce qui est imprimé ici?

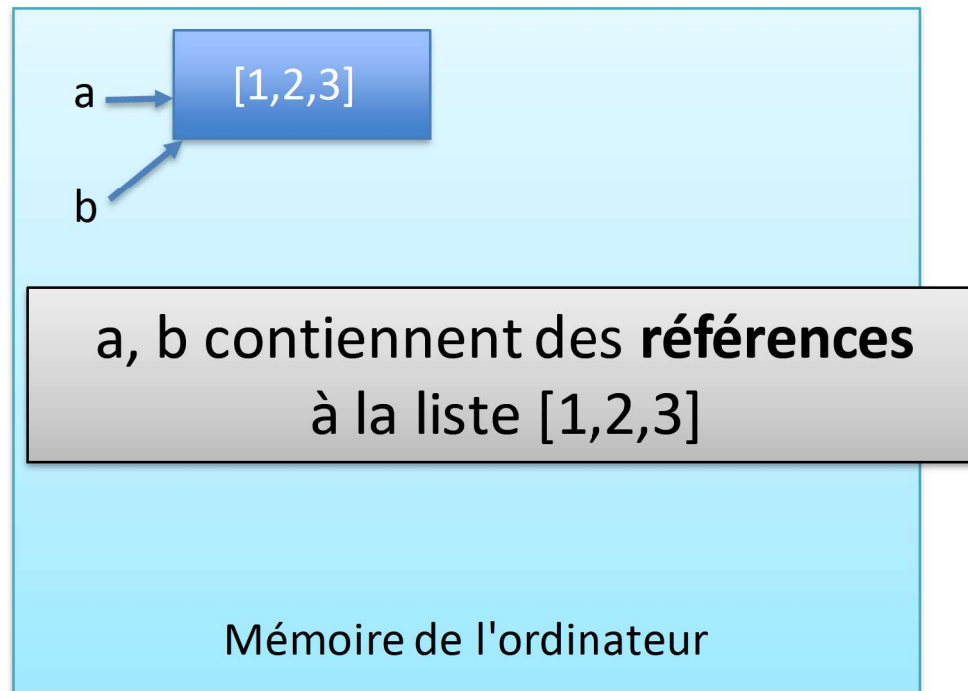
```
a = [1,2,3]
b = a
a.append ( 4 )
print(a)
print(b)
```

```
[1,2,3,4]
[1,2,3,4]
```

Modification de Listes

Qu'est-ce qui est imprimé ici?

```
a = [1,2,3]
b = a
a.append ( 4 )
print(a)
print(b)
```

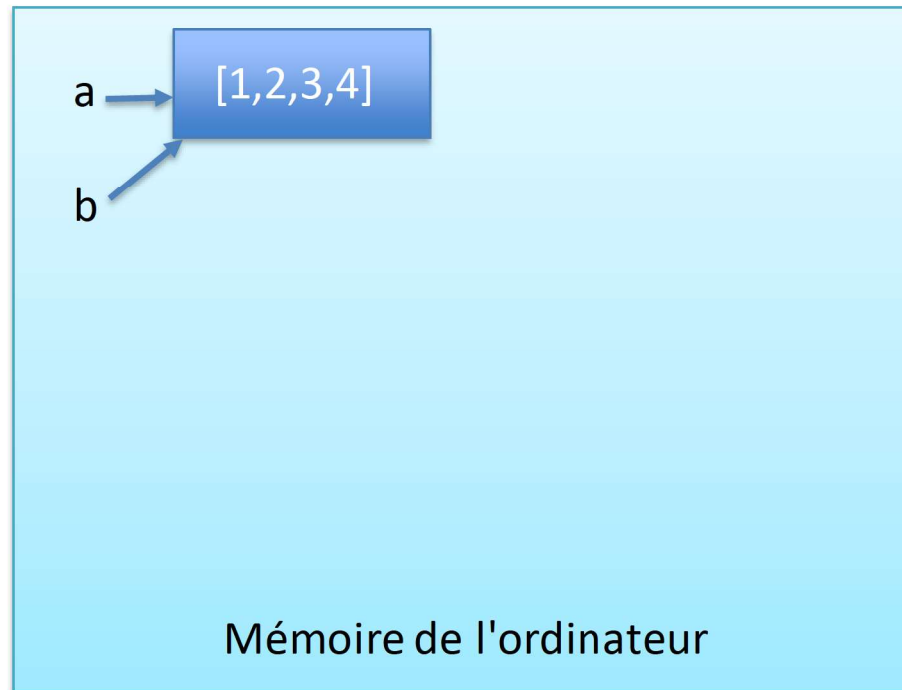


```
[1,2,3,4]
[1,2,3,4]
```

Modification de Listes

Qu'est-ce qui est imprimé ici?

```
a = [1,2,3]
b = a
a.append ( 4 )
print(a)
print(b)
```



```
[1,2,3,4]
[1,2,3,4]
```

Modification de Listes

Qu'est-ce qui est imprimé ici?

```
a = [1,2,3]
b = a
a.append ( 4 )
print(a)
print(b)
```

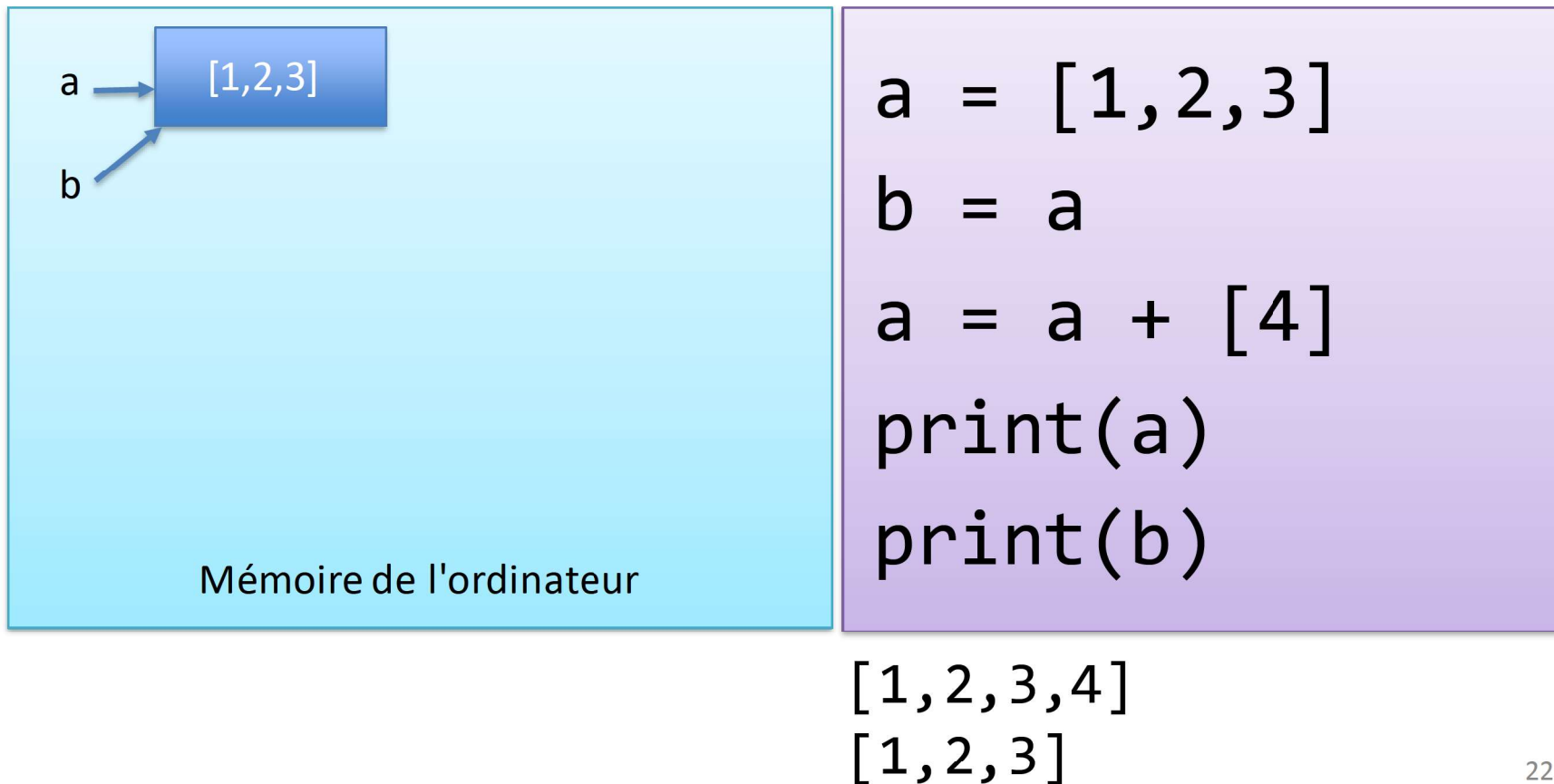
```
[1,2,3,4]
[1,2,3,4]
```

```
a = [1,2,3]
b = a
a = a + [4]
print(a)
print(b)
```

```
[1,2,3,4]
[1,2,3]
```

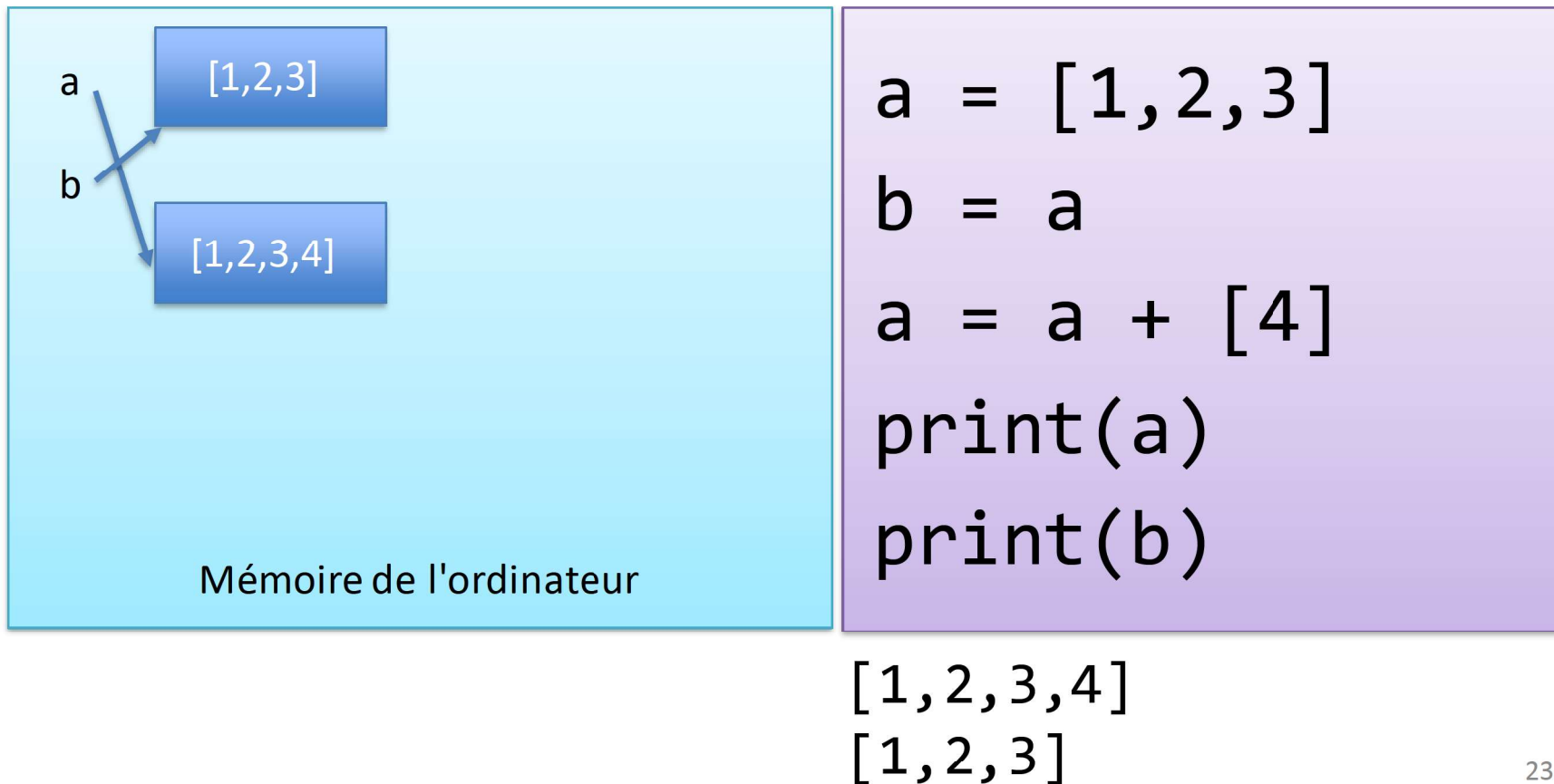
Modification de Listes

Qu'est-ce qui est imprimé ici?



Modification de Listes

Qu'est-ce qui est imprimé ici?



Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```

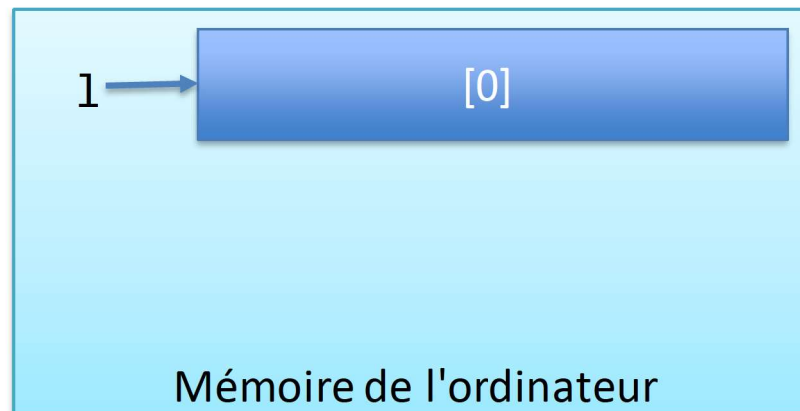


Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```

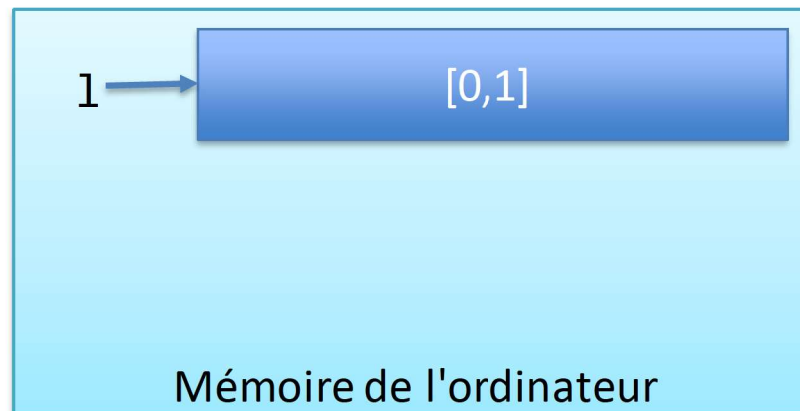


Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Modification de Listes

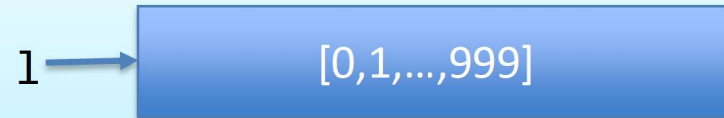
Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



Mémoire de l'ordinateur

Modification de Listes

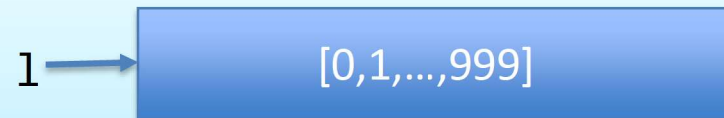
Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



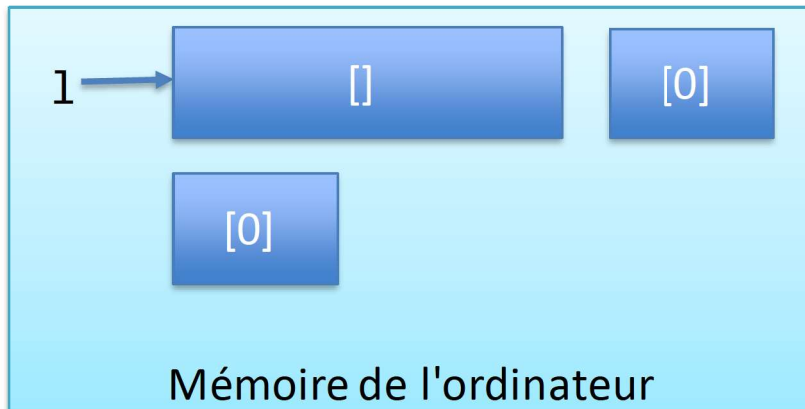
Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

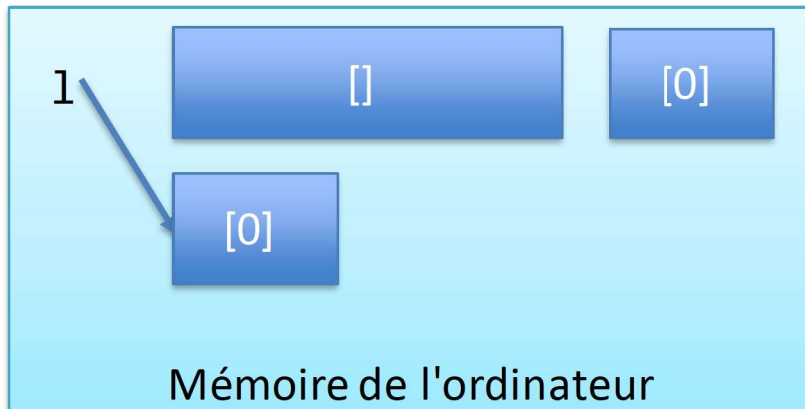
```
l = []  
for i in range(1000):  
    l.append ( i )
```



Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```



```
l = []  
for i in range(1000):  
    l.append ( i )
```

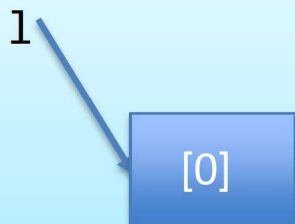


Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



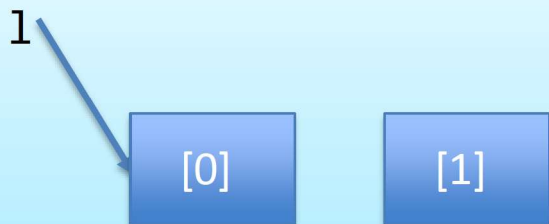
Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



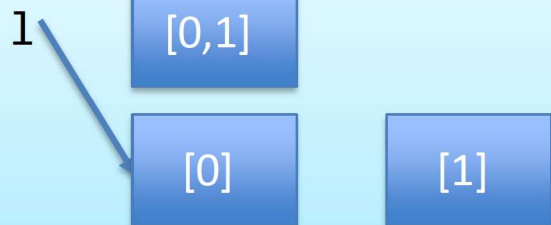
Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



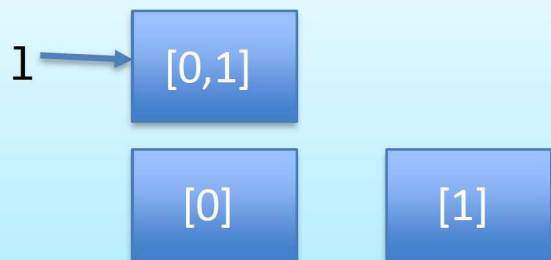
Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Mémoire de l'ordinateur



Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```

l → [0,1]

Mémoire de l'ordinateur

l → [0,1,...,999]

Mémoire de l'ordinateur

Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```



Modification de Listes

Quelle est la différence entre:

```
l = []  
for i in range(1000):  
    l = l + [i]
```

```
l = []  
for i in range(1000):  
    l.append ( i )
```

1 → [0,1,...,999]

LENT!

Mémoire de l'ordinateur

1 → [0,1,...,999]

RAPIDE!

Mémoire de l'ordinateur

Opérateurs sur Chaînes de Caractères

Similaire pour les chaînes de caractères

```
a = ""  
b = a  
for i in range(1000):  
    a = a + "*"
```

```
>>>> print(b)
```



Chaîne de caractères vide

```
>>>>
```

Modification de Tuples?

Quelle est la différence entre:

```
l = ( [1,2], [3,4] )  
l[0].append (3)  
print(l)
```

`([1,2,3],[3,4])`

```
l = ( [1,2], [3,4] )  
l[0] = l[0] + [3]  
print(l)
```

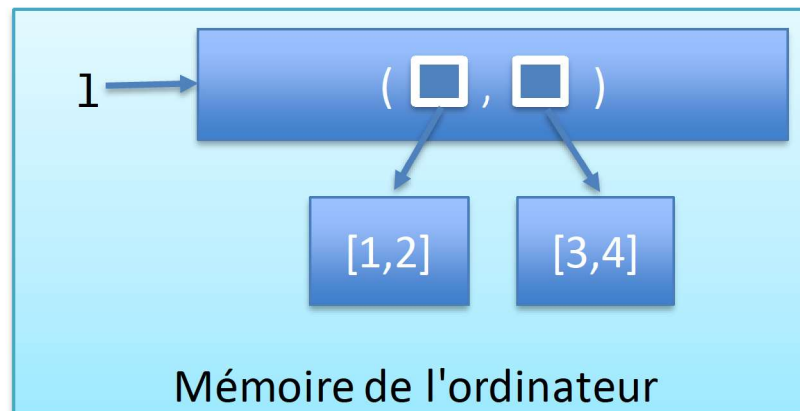
Erreur!

Modification de Tuples?

Quelle est la différence entre:

```
l = ( [1,2], [3,4] )  
l[0].append (3)  
print(l)
```

```
l = ( [1,2], [3,4] )  
l[0] = l[0] + [3]  
print(l)
```

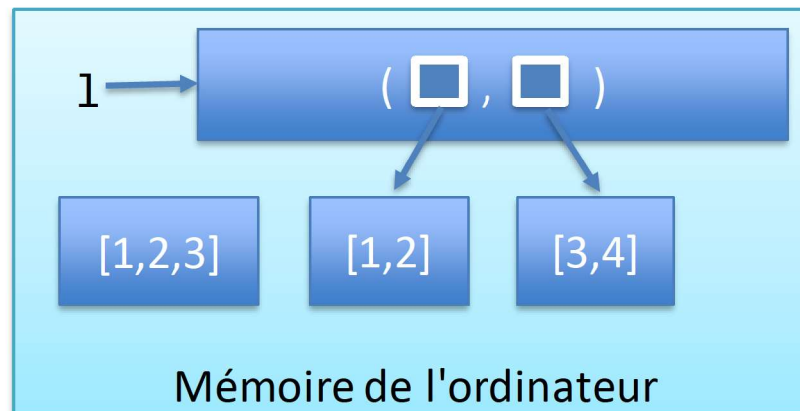


Modification de Tuples?

Quelle est la différence entre:

```
l = ( [1,2], [3,4] )  
l[0].append (3)  
print(l)
```

```
l = ( [1,2], [3,4] )  
l[0] = l[0] + [3]  
print(l)
```

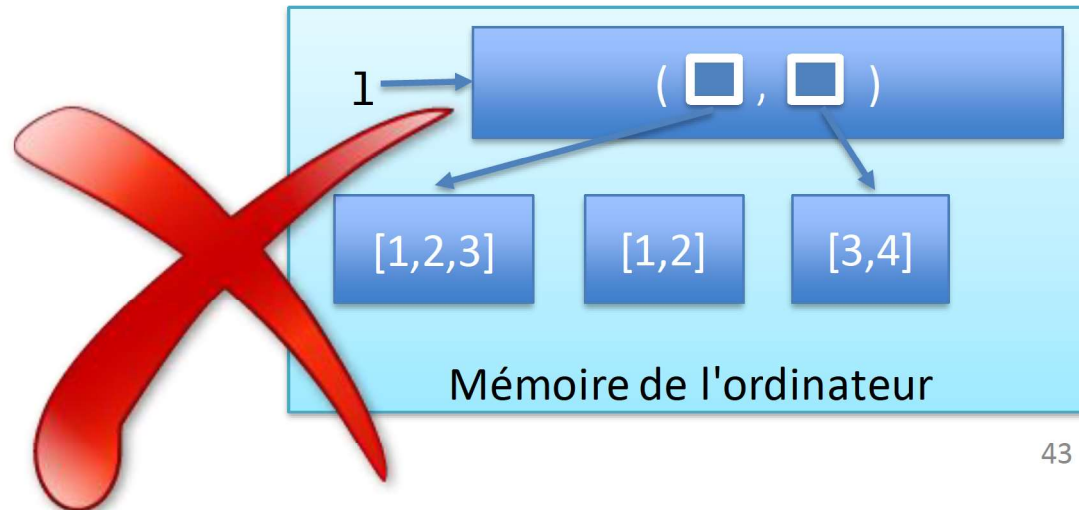


Modification de Tuples?

Quelle est la différence entre:

```
l = ( [1,2], [3,4] )  
l[0].append (3)  
print(l)
```

```
l = ( [1,2], [3,4] )  
l[0] = l[0] + [3]  
print(l)
```

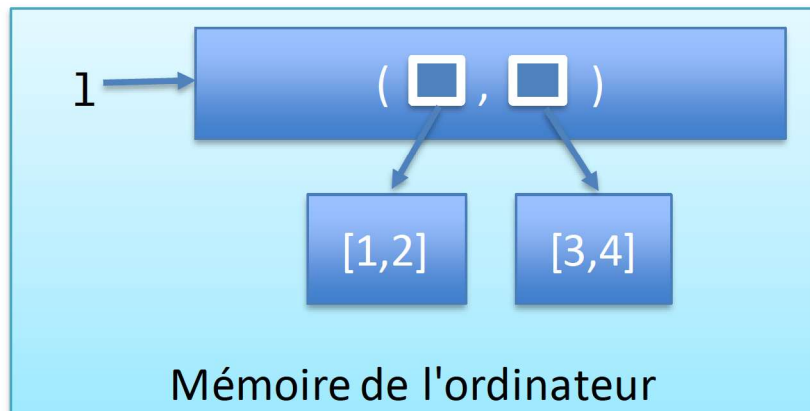


Modification de Tuples?

Quelle est la différence entre:

```
l = ( [1,2], [3,4] )  
l[0].append (3)  
print(l)
```

```
l = ( [1,2], [3,4] )  
l[0] = l[0] + [3]  
print(l)
```

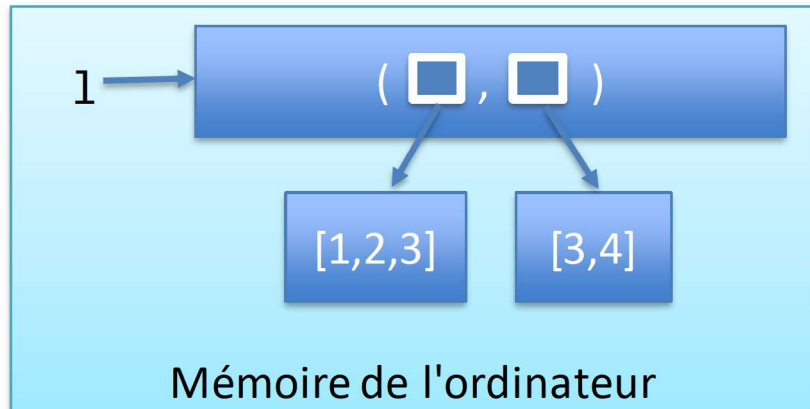


Modification de Tuples?

Quelle est la différence entre:

```
1 = ( [1,2], [3,4] )  
1[0].append (3)  
print(1)
```

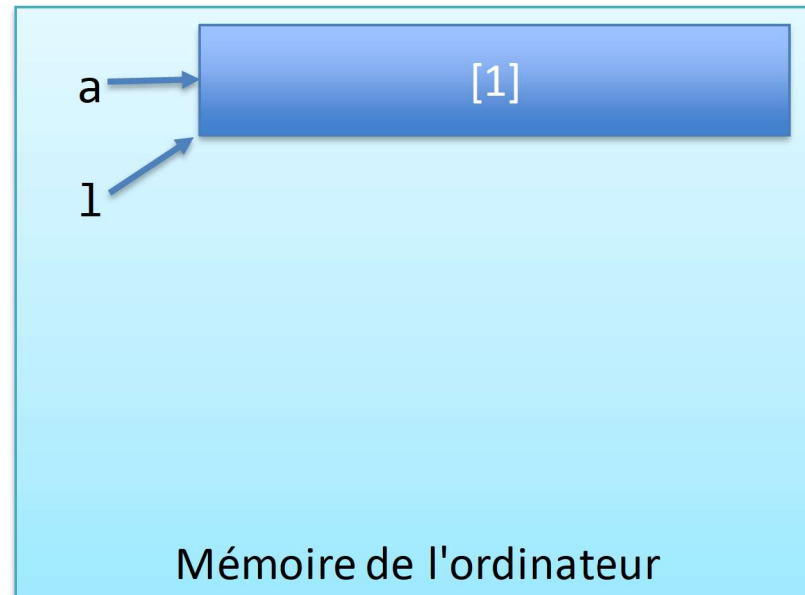
```
1 = ( [1,2], [3,4] )  
1[0] = 1[0] + [3]  
print(1)
```



Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l.append ( 2 )  
    l.append ( 3 )  
  
a = [1]  
modifier ( a )  
print(a)
```

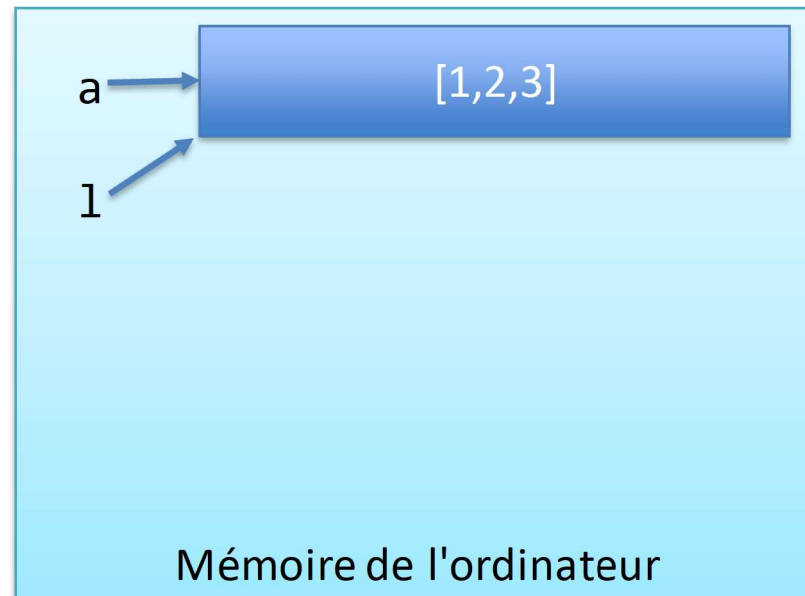


`[1, 2, 3]`

Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l.append ( 2 )  
    l.append ( 3 )  
  
a = [1]  
modifier ( a )  
print(a)
```



`[1,2,3]`

Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l.append ( 2 )  
    l.append ( 3 )  
  
a = [1]  
modifier ( a )  
print(a)
```



[1, 2, 3]

Fonctions

- Que fait ce code?

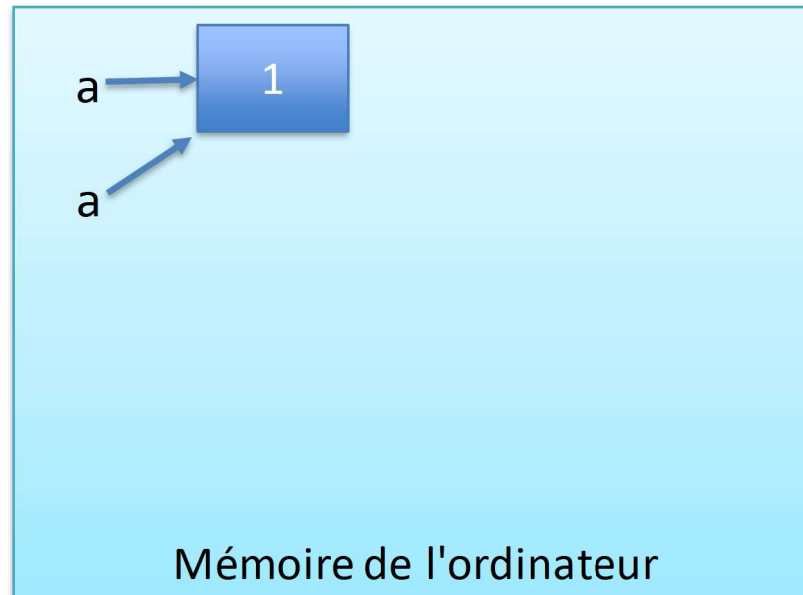
```
def modifier ( a ):  
    a = 2  
  
a = 1  
modifier ( a )  
print(a)
```

1

Fonctions

- Que fait ce code?

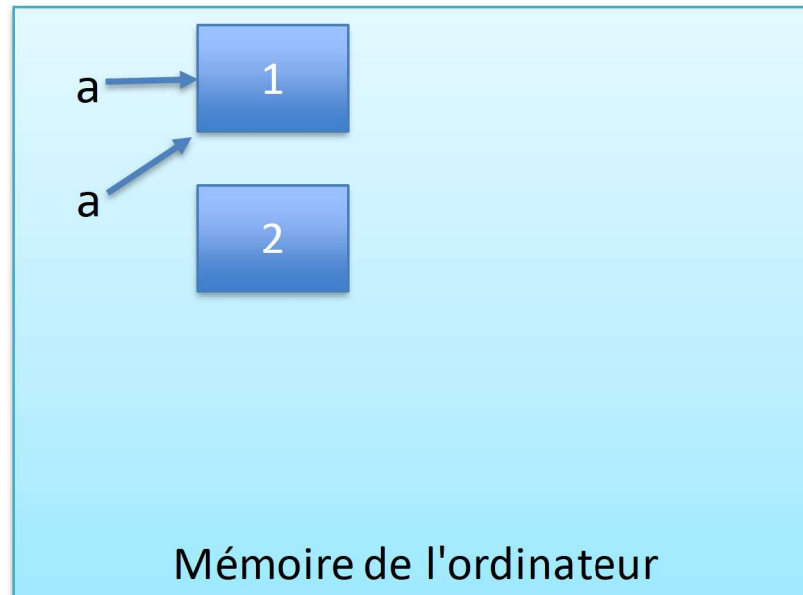
```
def modifier ( a ):  
    a = 2  
  
a = 1  
modifier ( a )  
print(a)
```



Fonctions

- Que fait ce code?

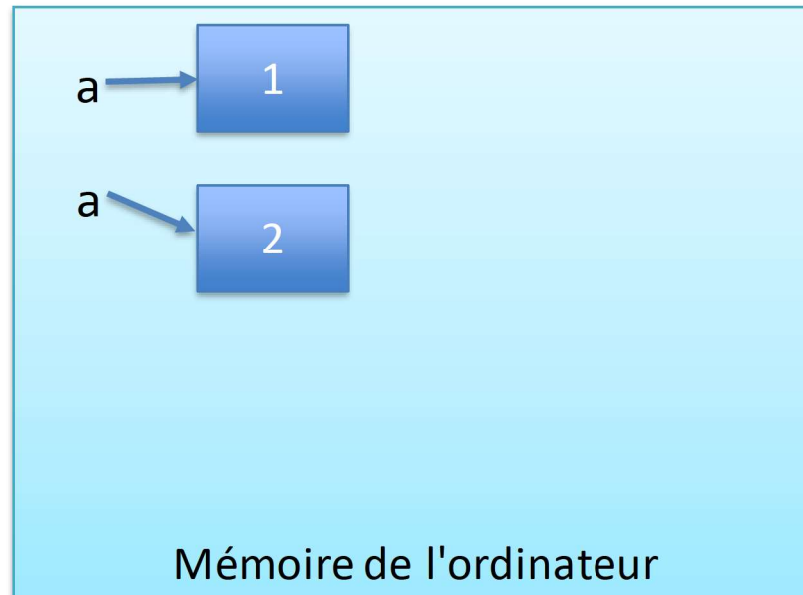
```
def modifier ( a ):  
    a = 2  
  
a = 1  
modifier ( a )  
print(a)
```



Fonctions

- Que fait ce code?

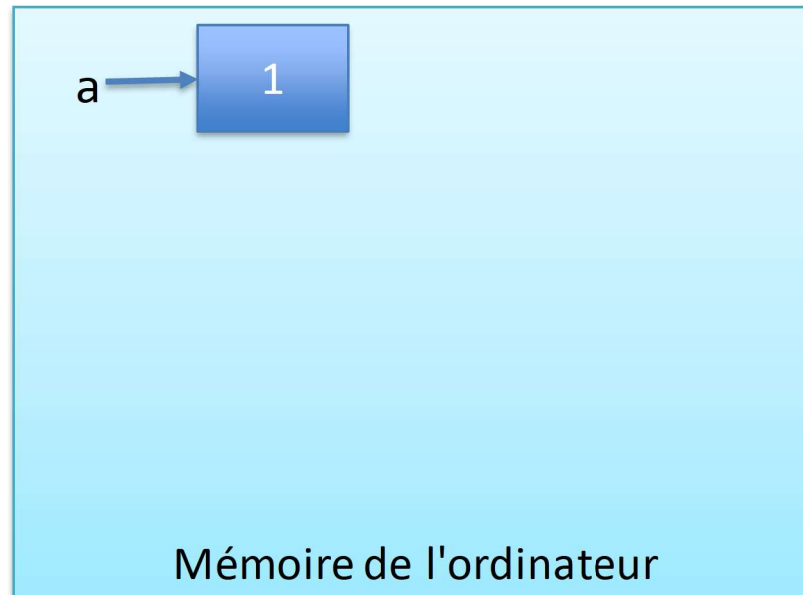
```
def modifier ( a ):  
    a = 2  
  
a = 1  
modifier ( a )  
print(a)
```



Fonctions

- Que fait ce code?

```
def modifier ( a ):  
    a = 2  
  
a = 1  
modifier ( a )  
print(a)
```

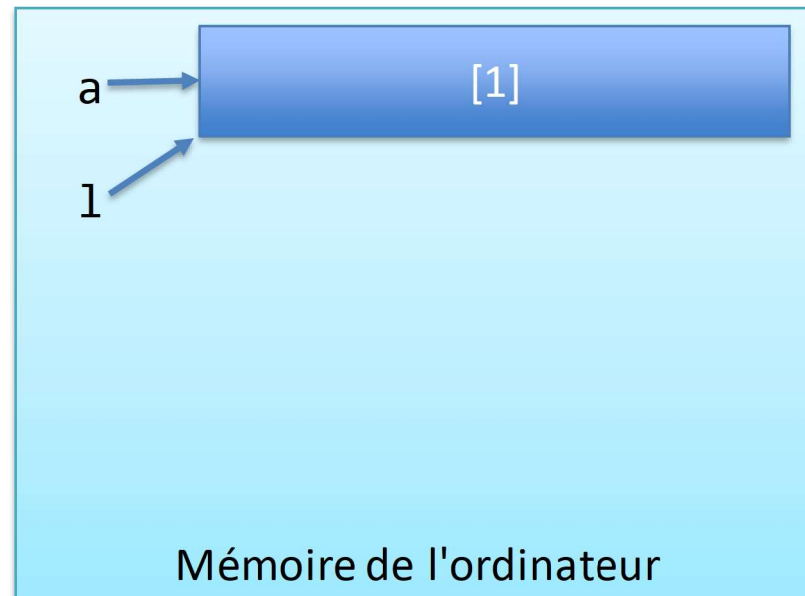


1

Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l = l + [2]  
    l = l + [3]  
  
a = [1]  
modifier ( a )  
print(a)
```

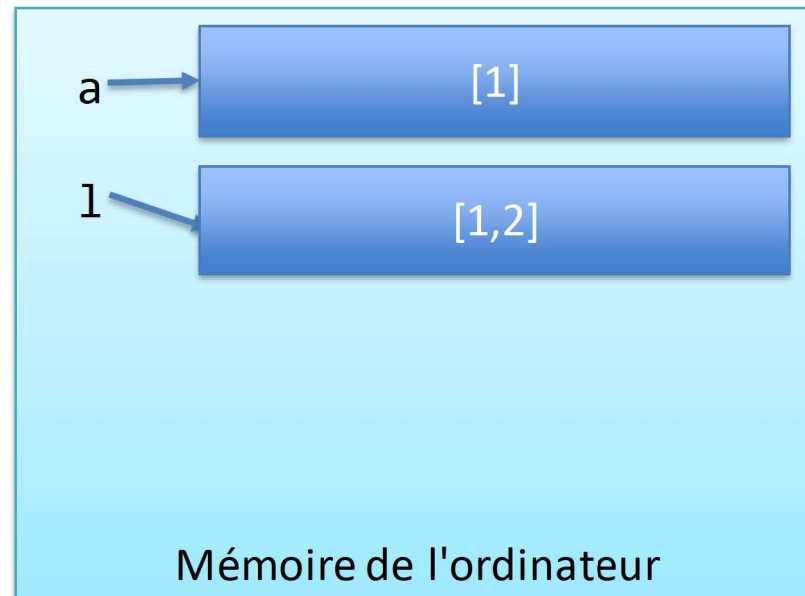


`[1]`

Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l = l + [2]  
    l = l + [3]  
  
a = [1]  
modifier ( a )  
print(a)
```

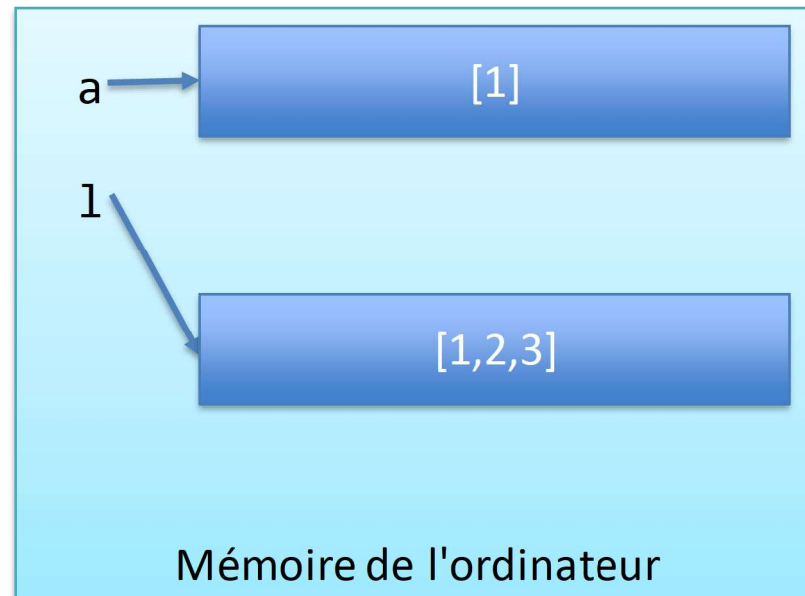


[1]

Fonctions

- Que fait ce code?

```
def modifier ( l ):  
    l = l + [2]  
    l = l + [3]  
  
a = [1]  
modifier ( a )  
print(a)
```



[1]

is vs ==

- Quelle est la différence ?

```
a = [1,2]
b = [1,2]
if a == b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai

```
a = [1,2]
b = [1,2]
if a is b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Faux

On trouve cela parfois dans
des programmes en Python

is vs ==

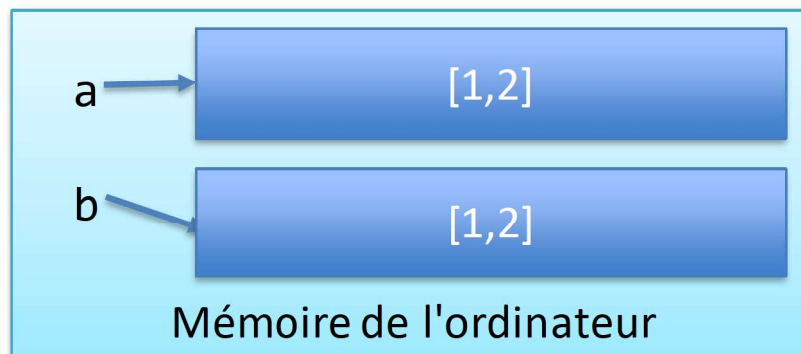
- Quelle est la différence ?

```
a = [1,2]
b = [1,2]
if a == b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai

```
a = [1,2]
b = [1,2]
if a is b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Faux



is vs ==

- Quelle est la différence ?

```
a = [1,2]
b = a
if a == b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai

```
a = [1,2]
b = a
if a is b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai

is vs ==

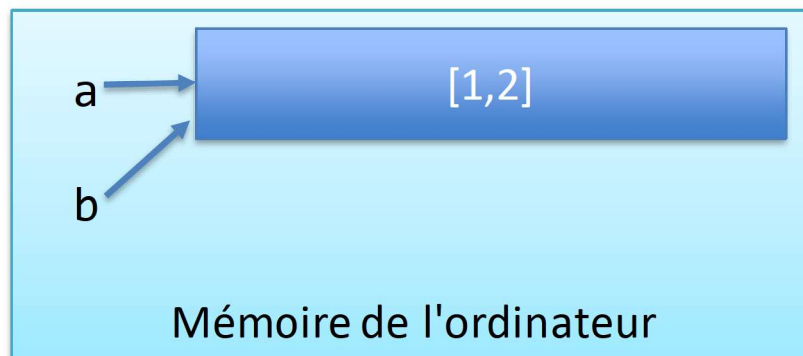
- Quelle est la différence ?

```
a = [1,2]
b = a
if a == b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai

```
a = [1,2]
b = a
if a is b:
    print ( "Vrai" )
else:
    print ( "Faux" )
```

Vrai



Enlever des Éléments

- Basé sur index:

```
a = ["Bruxelles", "Louvain", "Mons"]  
del a[1]  
print(a)
```

["Bruxelles", "Mons"]

- Basé sur valeur:

```
a = ["Bruxelles", "Louvain", "Mons"]  
a.remove("Mons")  
print(a)
```

["Bruxelles", "Louvain"]